Agentic FedGrid: Cost-Aware Edge Agents with Low-Rank Federated Adaptation for Network Intrusion Detection

Mohith Kumar M
Dept. of Information Science and Engineering
JSS Academy of Technical Education
Bengaluru, Karnataka, India
1725mohith@gmail.com

Anusha B
Dept. of Information Science and Engineering
JSS Academy of Technical Education
Bengaluru, Karnataka, India
anushabalaraju515@gmail.com

***Abstract*—Network intrusion detection at the edge must adapt to attacks that only some sites have seen, keep raw traffic on the device, and avoid letting one bad model update or one false alarm disrupt legitimate hosts. We present Agentic FedGrid, a system in which autonomous agents on edge nodes detect, respond to, and learn from network flows. A frozen tabular Transformer is adapted through low-rank (LoRA) adapters federated over a serverless AWS control plane, at 10.3 KB per upload instead of 103.7 KB for the full model. Each agent chooses actions by expected cost, escalates blocks gradually with automatic expiry, learns from analyst rollbacks, shares and retracts threat intelligence with its peers, and adopts a new global model only if it does not degrade its own held-out performance. Experiments ran on four AWS EC2 nodes with UNSW-NB15 and ToN_IoT over multiple seeds. Federation raises recall on an attack class withheld from pretraining from 41.1% to 54.5%, but moving only the frozen model's threshold to the same false-positive rate gives 53.4%, and the class AUROC does not improve (0.823 vs. 0.818). This holds even with the class spread over three clients or with five times longer local training, so default-threshold recall overstates federated gains. Under a model-replacement attack, FedAvg collapses to 47.5% accuracy, while median aggregation holds 86.5% and the adoption gate rejects every poisoned model. The full agent cuts benign traffic dropped by blocking from 49.8% to 12.5% on ToN_IoT and from 22.1% to 2.1% on UNSW-NB15, while letting fewer attacks through. The backbone reaches 87.7% accuracy on the official UNSW-NB15 test set and 99.3% on ToN_IoT, with 1.7 ms inference per flow.**

***Index Terms*—Federated learning, intrusion detection, LoRA, tabular Transformer, agentic AI, model poisoning, AWS serverless.**

I. Introduction

A. Background

Networks of connected devices at the edge, such as IoT gateways, branch offices and industrial controllers, now carry a growing share of traffic and an even faster-growing share of attacks [1]. Machine-learning intrusion detection systems (IDS) report high accuracy on benchmark datasets, using classical ensembles [2] or hybrid models that combine anomaly detection with supervised classification [3]. These systems share one assumption: all traffic is collected at a central server for training. At the edge this assumption is costly. Raw flows reveal which hosts talk to which, and moving them off-site raises privacy and legal concerns, consumes bandwidth, and adds latency to every model update. Many sites cannot share raw traffic at all.

B. Motivation

Federated learning (FL) [4] addresses part of this problem: each site trains on its own data and shares only model updates. Federated IDS have been built with semi-supervised objectives [5] and with Transformer backbones [6]. Four gaps remain. First, most federated IDS exchange the full model in every round, so communication grows with model size. Parameter-efficient methods such as LoRA [7] reduce this, but they are rarely applied to network-flow detection. Second, attack classes are unevenly spread: an attack may be labelled at only one site, and it is unclear whether federation actually transfers that knowledge to the other sites. Third, a single malicious client can replace the global model, and every site that adopts it blindly inherits the damage. Fourth, most work stops at classification. A verdict is useful only once it becomes an action, and an IDS that blocks a host on a single false positive cuts off all of that host's later legitimate traffic. Agentic approaches to security, which plan, act and adapt rather than only label [8], are mostly discussed separately from federated detection.

A further, methodological gap concerns evaluation. Federated IDS gains are usually reported as accuracy or recall at the default decision threshold. A rise in recall that comes together with a rise in false positives may reflect only a shift of the operating point. Moving the threshold of the original model would produce the same rise without any federation. Without threshold-free metrics, the two cannot be told apart.

C. Problem Statement

We consider K edge sites, each holding a private, non-IID set of labelled network flows, with one attack class labelled at only one site. The goal is to build a system that (i) adapts a shared detector without moving raw flows, (ii) keeps each model upload small, (iii) remains safe when a client

is malicious, and (iv) turns each verdict into an action that stops attacks without disrupting legitimate hosts. We call the edge component *agentic* in a specific, limited sense. It chooses among actions by expected cost, reverses its own blocks when they prove wrong, learns from analyst feedback, shares and retracts threat intelligence with its peers, and verifies a new global model before adopting it. It is therefore more than a classifier followed by a fixed rule. We also ask whether the federated detector genuinely learns to separate the new attack class, or only moves its decision threshold.

D. Contributions

This paper presents Agentic FedGrid and makes the following contributions:

1) **A deployed edge–cloud architecture.** It federates only the low-rank (LoRA) adapters of a frozen tabular Transformer through serverless AWS services (API Gateway, Lambda, DynamoDB, S3). Each upload is 10.3 KB, compared with 103.7 KB for the full model, and raw flows never leave the device. The live backend closed 12,610 federated rounds over 25 days, with straggler handling for partial participation.
2) **An edge agent whose actions go beyond classification.** It provides cost-aware action selection, graded and self-expiring blocks, analyst rollback with learned exceptions, peer threat intelligence with retraction, and verified adoption of global models. We evaluate it with both an ideal and an imperfect analyst.
3) **A threshold-free evaluation of federated adaptation.** For an attack class withheld from pretraining, the recall gain from federation is fully explained by a shift of the operating point. This holds even when the class is spread over three clients or when local training is five times longer. Default-threshold recall is therefore an unreliable measure of federated gains.
4) **A multi-seed evaluation on real cloud hardware.** It covers LoRA rank versus full-model federation; FedAvg, FedProx and median aggregation; model poisoning; the local training components; and the agent's decision layer, together with gradient-boosted and centrally trained reference models.

E. Paper Organisation

Section II reviews related work. Section III describes the system model and methodology, and Section IV the experimental setup. Section V reports the results. Section VI discusses limitations, and Section VII concludes with directions for future work.

II. Related Work

A. Machine-Learning Intrusion Detection

Machine-learning IDS are commonly evaluated on public benchmarks such as CICIDS2017, UNSW-NB15 [9] and ToN_IoT [10]. Pandit et al. [2] compare classical classifiers and ensemble learning on CICIDS2017. Bayesh and Hossain [3] combine a Gaussian mixture model, which flags previously unseen anomalies, with supervised models that identify known attack types. Both approaches train centrally on pooled data and report results at a fixed decision threshold. Benchmark results are also often obtained on random splits of a single file, which are easier than the official train/test separation; we show the size of this gap on UNSW-NB15 in Section V.

B. Federated Intrusion Detection

Federated learning [4] trains a shared model while each client keeps its data. Lim et al. [11] survey FL at the network edge and identify communication cost, resource allocation, and privacy and security as its main implementation challenges. Lameh et al. [12] study FL on edge devices empirically and show how accuracy and cost depend on the number of clients and rounds. For intrusion detection, FLUIDS [5] combines federated learning with a semi-supervised design: an autoencoder learns from unlabelled traffic and a supervised component from the labelled part. Mothukuri et al. [13] train recurrent (GRU) detectors for IoT attacks in federated rounds, so raw device data stays local. Ruzafa-Alcázar et al. [14] evaluate differential privacy in a federated IDS for the industrial IoT on non-IID ToN_IoT data and compare FedAvg with Fed+. Devi et al. [6] convert traffic records into two-dimensional images, classify them with a lightweight Vision Transformer, and combine federated adaptive training with contrastive adversarial training. Vyas et al. [1] survey privacy-preserving federated IDS for IoT, covering homomorphic encryption, differential privacy and secure multiparty computation. These systems federate the full model in every round and report accuracy at the default threshold. Non-IID data, where clients see different attack classes, is known to slow and destabilise FedAvg; FedProx [15] adds a proximal term to the local objective to limit client drift.

C. Communication-Efficient and Robust Federation

LoRA [7] freezes a pretrained network and trains low-rank updates of selected weight matrices. In FL this shrinks each upload to a small fraction of the model. It is widely used for language models but has rarely been applied to network-flow intrusion detection. Federation also opens an attack surface. A malicious client can scale its update so that it replaces the global model [16]. Robust aggregators such as the coordinate-wise median [17] bound the influence of any single client, and Lyu et al. [18] survey the resulting space of poisoning attacks and defences. Most federated IDS work does not evaluate either defence, nor what an individual site should do if the server itself distributes a poisoned model.

D. Agentic and Automated Response

Panwar and Abdelrahman [8] review agentic AI in cybersecurity. Such systems perceive, decide, act and adapt rather than only label. The review identifies autonomous response, adaptation to new threats and the explainability of black-box models as open problems. Hammar and Stadler [19] formulate intrusion prevention as an optimal multiple-stopping problem,

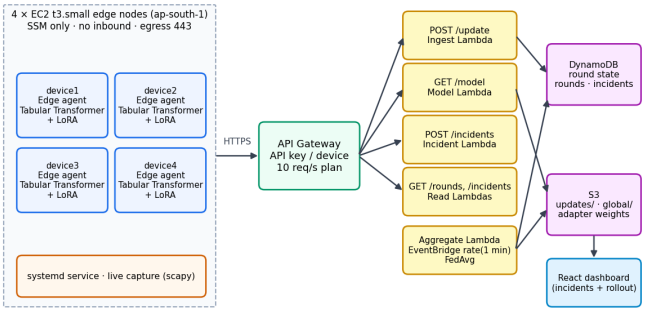


Fig. 1. Agentic FedGrid architecture: four EC2 edge agents (perceive–decide–act–reflect) exchange only LoRA adapters and incident metadata with a serverless AWS control plane (API Gateway, Lambda, DynamoDB, S3).

show that optimal defender policies have a threshold structure, and learn them with reinforcement learning on a digital twin of a 31-server infrastructure. Attention-based attribution [20] is one practical route to explaining Transformer decisions. In federated IDS, however, response is usually treated separately from detection: the cost of a wrong block, its reversibility, and the effect of human feedback on later decisions are seldom measured.

E. Research Gap

Table I summarises the surveyed work. To our knowledge, no prior work combines parameter-efficient federation of a network-flow IDS with an edge agent that acts on, reverses and learns from its own verdicts. We also found no federated IDS evaluation that tests whether a recall gain reflects better class separation or only a shifted decision threshold. Agentic FedGrid addresses both gaps.

III. System Model and Methodology

A. Threat model and setting

K=4 edge nodes (AWS EC2 t3.small, 2 vCPU, 2 GB RAM) each observe network flows and hold a private, non-IID labelled partition (Dirichlet α=0.5 over attack categories). Devices 1–2 hold UNSW-NB15 flows and devices 3–4 hold ToN_IoT flows. One UNSW-NB15 category (Fuzzers) is withheld from central pretraining and from every device except device 1, emulating an attack that only one site has labelled examples of. Raw flows never leave a device; only adapter weights and incident metadata cross the network. Coordination runs on AWS serverless services (API Gateway, Lambda, DynamoDB, S3); nodes authenticate with per-device API keys. Fig. 1 shows the architecture.

B. Flow representation and backbone

Each flow is mapped to a cross-dataset canonical schema of five continuous features (duration, source/destination bytes, source/destination packets) and three categorical features (protocol, service, connection state). Continuous features are transformed as $\tilde{x}_j = (\log(1+x_j) - \mu_j)/\sigma_j$ with μ_j, σ_j frozen centrally so all devices share one encoding. Backbone v2 embeds every continuous feature as its own token, $\mathbf{t}_j = \tilde{x}_j \mathbf{w}_j + \mathbf{b}_j$ ($\mathbf{w}_j, \mathbf{b}_j \in \mathbb{R}^d$), and every categorical feature through an embedding table, giving n=8 tokens of width d=32. Backbone v1 instead projects all five continuous features into a single token (n=4). A two-layer Transformer encoder (4 heads, FFN width $4d$) is followed by mean pooling and a linear head. Unless stated otherwise, all federated, ablation and agent experiments use backbone v1; v2 is evaluated as a backbone and in one federated replication (Section V-B).

C. Parameter-efficient federated adaptation

After central pretraining the backbone is frozen. Each FFN projection $W_0 \in \mathbb{R}^{m\times n}$ receives a low-rank update

$$W = W_0 + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r\times n},\ B \in \mathbb{R}^{m\times r}, \tag{1}$$

with r=4, α=8, and B initialised to zero. Only $\{A, B\}$ and the classification head are trained and communicated (≈2.6K parameters, 10.3 KB of float32 per upload). The server aggregates with FedAvg [4],

$$\theta^{t+1} = \sum_{k\in S_t} \frac{n_k}{\sum_{j\in S_t} n_j}\, \theta_k^t, \tag{2}$$

where S_t is the set of devices that reported in round t. A round closes when all K devices report or, via a scheduled straggler handler, after a timeout with at least one participant. FedProx [15] is evaluated as an alternative local objective, $\ell_k(\theta) + \frac{\mu}{2}\|\theta - \theta^t\|^2$, μ=0.01.

D. Local training

Each local update runs a fixed budget of $E\cdot S = 5\times 20$ mini-batch steps (batch 64), independent of buffer size. Minibatches are sampled with probability proportional to a clipped per-example loss, $w_i = \min(\ell_i, 3) + 0.3$, recomputed each epoch (online hard-example mining). Each step minimises clean plus FGSM-perturbed [21] cross-entropy,

$$\begin{aligned}\mathcal{L} = {} & \mathrm{CE}(f(x), y) \\ & + \mathrm{CE}\big(f(x + \epsilon\,\mathrm{sign}(\nabla_x \mathrm{CE}(f(x), y))), y\big),\end{aligned} \tag{3}$$

with ϵ=0.05, applied to the normalised continuous features. A shuffled 15% of each local buffer is held out for the device's own validation.

E. The edge agent

Each node runs a perceive–decide–act–reflect loop. Unlike a fixed pipeline, the agent chooses between actions under uncertainty, verifies what it receives, and adapts from feedback.

Cost-aware action selection. For a flow with $p = P(\text{attack} \mid x)$ the agent takes

$$\begin{aligned}a^* = \arg \min_{a\in\{\text{NOOP},\text{ALERT},\text{BLOCK}\}} & (1-p)\, C(a, \text{benign}) \\ & + p\, C(a, \text{attack}),\end{aligned} \tag{4}$$

with default costs $C(\text{NOOP}) = (0, 10)$, $C(\text{ALERT}) = (1, 3)$, $C(\text{BLOCK}) = (20, 0)$ (benign, attack). These are deployment assumptions, not learned values, and must be set per site. The original rule (BLOCK iff $p \geq 0.9$) is kept as a baseline.

TABLE I
COMPARISON WITH SURVEYED WORK.

Work	Method	Data	Federated	Param.-efficient	Response/agent	Threshold-free eval.
Pandit et al. [2]	ML ensembles	CICIDS2017	–	–	–	–
Bayesh & Hossain [3]	GMM + supervised	CICIDS2017	–	–	–	–
Lameh et al. [12]	FL on edge devices	non-IDS	✓	–	–	–
Aouedi et al. [5]	Semi-supervised FL	IDS benchmark	✓	–	–	–
Mothukuri et al. [13]	Federated GRU	IoT traffic	✓	–	–	–
Ruzafa-Alcázar et al. [14]	FL + differential privacy	ToN_IoT	✓	–	–	–
Devi et al. [6]	Federated ViT + adv. training	IDS benchmark	✓	–	–	–
Hammar & Stadler [19]	RL intrusion prevention	Digital twin	–	–	✓	–
Panwar & Abdelrahman [8]	Review (agentic AI)	–	–	–	discussed	–
Agentic FedGrid	LoRA-federated Transformer + agent	UNSW-NB15, ToN_IoT	✓	✓	✓	✓

Graded response. A block on source IP s raises its level λ_s. Levels $\lambda_s < 4$ are temporary with TTL $60 \cdot 2^{\lambda_s - 1}$ s, and level 4 is permanent. Expired blocks lift automatically.

Analyst feedback. A block an analyst marks as a false positive is rolled back. The host gets a 30-minute exception (block verdicts downgraded to alerts), and the benign-side block cost is updated multiplicatively, $C(\text{BLOCK}, \text{benign}) \leftarrow \min(1.1\,C, 100)$; a confirmed block lowers it by 5% (floor 0.1).

Shared threat intelligence with retraction. Every 15 s an agent pulls recent BLOCK/ROLLBACK records of its peers from the cloud incident store. An IP whose latest record is a peer's block is escalated faster: alerts become blocks, starting one level higher. A rollback by any device retracts the IP everywhere.

Verified model adoption. When a new global model θ^{t+1} arrives, the agent evaluates it and its current model on its local held-out set and adopts it only if

$$\begin{aligned} &\mathrm{Acc}(\theta^{t+1}) \geq \mathrm{Acc}(\theta_k) - 0.01 \wedge \\ &\mathrm{FPR}(\theta^{t+1}) \leq \mathrm{FPR}(\theta_k) + 0.02. \end{aligned} \tag{5}$$

Otherwise it keeps its model and logs a rejection.

Autonomous retraining. A device retrains when the rolling mean of its top-class confidence over the last 50 flows falls below 0.6 (minimum 20 flows), or when its accuracy on its own labelled buffer falls below 0.90. Self-evaluation runs at most once per minute, and retraining at most once per 5 minutes.

Explanation. Every BLOCK/ALERT carries an attention-rollout attribution [20], $\hat{A}^{(l)} = 0.5A^{(l)} + 0.5I$ (row-normalised), $R = \hat{A}^{(L)} \cdots \hat{A}^{(1)}$, with token importance given by the column sums of R. This is a heuristic attribution, not a causal explanation. An optional LLM triage worker (Claude on Amazon Bedrock) summarises incidents for analysts asynchronously, off the detection path. It receives only incident metadata and is not evaluated in this paper.

IV. EXPERIMENTAL SETUP

All reported numbers were produced on the four EC2 t3.small nodes (ap-south-1). The federated round experiments execute the deployed Lambda handler code on the nodes, with DynamoDB and S3 replaced by moto mocks, so that rounds are reproducible and seeded. The ablation runs execute the same FedAvg arithmetic in-process. The live backend (API Gateway, Lambda, DynamoDB, S3, per-device API keys) was exercised separately by the deployed agents: between 21 September and 16 October 2026 it closed 12,610 rounds. Only 22 of them had all four devices; the scheduled straggler handler closed the rest with one to three participants. This validates the coordination path under real partial participation but is not used for any accuracy number.

Datasets. We use UNSW-NB15 [9] and ToN_IoT [10]. Test sets are never used for training or model selection: the official UNSW-NB15 testing set (82,332 flows; 6,062 Fuzzers) and a stratified 15% ToN_IoT split (31,656 flows). Backbone checkpoints are selected on a random 15% split of the pretraining pool.

Protocol and metrics. Each configuration is repeated over 3–4 seeds and reported as mean ± sample standard deviation. Besides default-threshold metrics (accuracy, false-positive rate, macro-F1, Fuzzers recall) we report threshold-free ones: AUROC, Fuzzers-vs-benign AUROC, and Fuzzers recall at a fixed benign FPR. The threshold for the last is the corresponding quantile of the model's scores on the benign test flows; it is an evaluation device and is never used by the deployed system.

Reference models. Gradient-boosted trees (scikit-learn `HistGradientBoosting` [22]) and a centrally trained Transformer use the same eight features and test sets.

Agent simulation. The real agent, actuator and policy code are driven on a simulated clock, with one flow every 0.05 s distributed round-robin over four agents. ToN_IoT keeps its real source IPs (47 hosts). UNSW-NB15's IPs are synthetic, so flows are grouped into synthetic sessions (300 benign hosts, 5 attacker IPs per category); this grouping is an assumption. The analyst reviews blocks after 300 s. In the main runs it reviews every block and is always correct, an upper bound on what feedback can achieve. A sensitivity study then has it review a random 50% of blocks with a 10% or 20% chance of a wrong verdict, and varies the learned-exception duration (30 vs. 5 minutes).

V. RESULTS

A. Backbone and reference models

Table II reports the frozen backbone before any federation. Moving from one token for all continuous features (v1) to one token per feature with a longer one-cycle schedule (v2) raises

TABLE II

FROZEN BACKBONE AND REFERENCE MODELS ON THE OFFICIAL UNSW-NB15 TEST SET (82,332 FLOWS) AND THE TON_IOT HELD-OUT SPLIT (31,656 FLOWS); SAME EIGHT FEATURES, EC2. GBM = GRADIENT-BOOSTED TREES. MEAN ± SD OVER 3 SEEDS WHERE SHOWN (V1 FROZEN: SINGLE RUN; FEDERATED ROW: 4 SEEDS).

Model	UNSW acc. (%)	UNSW FPR (%)	Fuzzers rec. (%)	ToN acc. (%)
GBM (Fuzzers excluded)	90.10 ± 0.17	8.92 ± 0.40	34.27 ± 0.25	99.55 ± 0.01
Centralized Transformer (Fuzzers incl.)	82.39 ± 0.45	36.38 ± 1.65	89.76 ± 1.74	98.88 ± 0.07
Backbone v1, frozen	87.19	14.31	41.06	98.84
Backbone v2, frozen	87.73 ± 0.35	12.94 ± 1.13	36.81 ± 0.81	99.28 ± 0.04
Backbone v1 + 4 LoRA rounds	86.46 ± 0.07	19.21 ± 0.61	54.50 ± 1.92	–

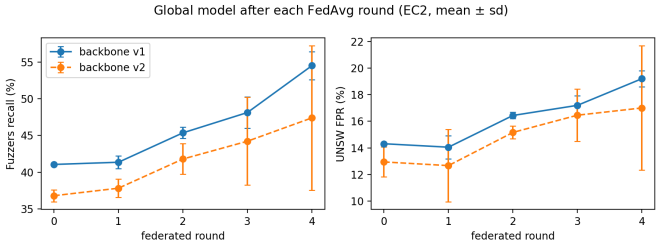


Fig. 2. Global model per FedAvg round on device 2 (never sees Fuzzers locally): Fuzzers recall at the default threshold (left) and UNSW-NB15 FPR (right), mean ± sd over seeds, EC2. Both rise together; moving the frozen backbone's threshold to the same FPR gives the same recall (Table IV).

UNSW-NB15 accuracy from 87.19% to 87.73±0.35%, lowers its FPR from 14.31% to 12.94±1.13%, and lifts ToN_IoT to 99.28±0.04% (FPR 1.64%). Inference stays at 1.7 ms per flow on a t3.small. The better general fit comes with lower recall on the withheld Fuzzers class before federation (36.81% vs. 41.06%), so v2 is not uniformly better.

Table II also places the backbone between two references trained on the same eight features. Gradient-boosted trees trained on the same pretraining pool reach 90.10±0.17% accuracy at 8.9% FPR, so the Transformer itself costs about 2.4 points on this split. We keep it because LoRA federation, FGSM training and attention attribution all require a differentiable model, and the tree's advantage does not extend to the unseen class (34.3% Fuzzers recall vs. 41.1%). The backbone scores 97.9% on a random split of its own pretraining pool. The roughly 10-point drop to the official test file is distribution shift between the official training and test files, not a lack of fit. A Transformer trained centrally on all device data, Fuzzers included, reaches 89.8% Fuzzers recall but only 82.4% accuracy at 36.4% FPR (3 seeds).

B. Federated adaptation to an unseen attack class

Fuzzers flows are excluded from pretraining and appear only on device 1, revealed 25/50/75/100% over four rounds. All metrics are measured on the global model on device 2, which never sees a Fuzzers flow.

At the default threshold (Fig. 2), Fuzzers recall rises from 41.06% to 54.50±1.92% (backbone v1, 4 seeds) and replay-set accuracy from 28.14% to 43.53%. Accuracy moves only from 87.19% to 86.46%, but the FPR rises from 14.31% to 19.21%. Backbone v2 gives 36.81% → 47.39±9.83% at 17.01% FPR (3 seeds).

TABLE III

THRESHOLD-FREE VIEW OF FEDERATION (OFFICIAL UNSW-NB15 TEST SET, 3 SEEDS; EC2): AUROC OVER ALL FLOWS, FUZZERS-VS-BENIGN AUROC, AND FUZZERS RECALL AT A FIXED 10% BENIGN FPR (THRESHOLD SET ON THE BENIGN TEST FLOWS), BEFORE AND AFTER 4 ROUNDS.

Variant (round 4 unless stated)	AUROC	Fuzzers AUROC	Fuzzers rec. @10% FPR (%)
Before federation (round 0)	0.962 ± 0.000	0.823 ± 0.000	37.99 ± 0.00
LoRA r=4 (default)	0.959 ± 0.001	0.818 ± 0.003	34.83 ± 1.40
LoRA r=8	0.958 ± 0.001	0.818 ± 0.003	34.40 ± 1.45
Full-model FedAvg	0.950 ± 0.003	0.809 ± 0.003	30.19 ± 0.51
Median aggregation	0.959 ± 0.001	0.817 ± 0.001	34.63 ± 0.98
Fuzzers on 2 clients	0.959 ± 0.001	0.818 ± 0.001	34.69 ± 0.60
Fuzzers on 3 clients	0.957 ± 0.000	0.818 ± 0.001	34.48 ± 0.31
5× local steps	0.956 ± 0.002	0.817 ± 0.001	33.77 ± 0.86
3 clients + 5× steps	0.826 ± 0.115	0.820 ± 0.006	36.12 ± 2.98

TABLE IV

FEDERATED FUZZERS RECALL VS. THE FROZEN BACKBONE WITH ONLY ITS DECISION THRESHOLD MOVED TO THE SAME BENIGN FPR (OFFICIAL UNSW-NB15 TEST SET; EC2). THE FROZEN MODEL NEEDS NO FEDERATION AT ALL.

Federated model	FPR (%)	Fuzzers rec. (%)	Frozen, threshold moved (%)
LoRA r=4, 4 rounds	18.28 ± 0.83	50.48 ± 3.16	51.62
LoRA r=4, Lambda path (4 seeds)	19.21 ± 0.61	54.50 ± 1.92	53.37
LoRA r=8, 4 rounds	21.24 ± 2.66	59.12 ± 6.45	56.96
Full-model FedAvg, 4 rounds	33.08 ± 1.80	78.04 ± 3.34	84.00
Fuzzers on 2 clients, 4 rounds	18.47 ± 2.19	51.15 ± 6.93	51.93
5× local steps, 4 rounds	28.24 ± 6.83	71.41 ± 11.17	71.68
Fuzzers on 3 clients, 4 rounds	33.56 ± 0.83	79.42 ± 1.40	85.09
3 clients + 5× steps, 4 rounds	37.36 ± 1.44	87.19 ± 4.38	89.69

Threshold-free (Tables III, IV), the picture changes. Across rounds, AUROC stays at 0.962 → 0.959, the Fuzzers-vs-benign AUROC at 0.823 → 0.818, and Fuzzers recall at a fixed 10% FPR falls from 38.0% to 34.8%. Moving only the frozen backbone's threshold to the FPR each federated variant ends at gives the same recall: 53.4% vs. 54.5% at 19.2% FPR, 51.6% vs. 50.5% at 18.3%, and 57.0% vs. 59.1% at 21.2%. Federation in this setting therefore moves the operating point toward "attack"; it does not teach the model to separate Fuzzers from benign traffic. The information is not absent. The centrally trained model reaches 0.847 Fuzzers AUROC and 48.1% recall at 10% FPR (one seed).

More clients and longer local training. Two obvious explanations are that only one of four clients holds the class and that each round runs only 100 local steps. We tested both on the same Fuzzers flows: split into disjoint shards over two or three clients, with 5× more local steps (500 per round), and both combined (Tables V, III, IV; 3 seeds each). Default-threshold recall rises to 51.1% (two clients), 71.4% (5× steps), 79.4% (three clients) and 87.2% (both), but the FPR rises with it, to 18.5%, 28.2%, 33.6% and 37.4%. The Fuzzers-vs-benign AUROC stays at 0.817–0.820 in every variant (0.823 before federation), and recall at 10% FPR stays at 33.8–36.1% (38.0% before). The frozen backbone with only its threshold moved to the same FPR matches or beats each variant: 51.9% vs. 51.1%, 71.7% vs. 71.4%, 85.1% vs. 79.4%, and 89.7% vs. 87.2%. Longer local training also costs robustness (FGSM accuracy 74.5% → 53.9%), and with three clients and 5× steps two of three seeds collapse (UNSW accuracy 59.8% and 59.6%, AUROC 0.77 and 0.75). Participation and local budget are therefore not the bottleneck in the regimes we tested. Together

with the full-model result below, which removes the capacity limit and degrades the ranking further, this suggests that the binary local objective itself moves the decision boundary rather than the representation. We regard default-threshold recall as an unreliable measure of federated gains and report both.

C. Ablations

Table V and Fig. 3 vary one component at a time. Unless stated otherwise, each variant uses backbone v1, LoRA $r{=}4$ and FedAvg, with 3 seeds.

Rank and full-model federation. Ranks 1, 2 and 4 behave almost identically: Fuzzers recall 50.5–52.4%, FPR $\approx$18.3%, at 2.8–10.3 KB per upload. Rank 8 (20.3 KB) reaches 59.1% at 21.2% FPR. Full-model FedAvg sends 10$\times$ more data (103.7 KB) and reaches 78.0% recall at 33.1% FPR. Threshold-free, the full model is worse: AUROC 0.950 vs. 0.959 for LoRA, 30.2% vs. 34.8% Fuzzers recall at 10% FPR, and 78.0% vs. 84.0% for the frozen backbone at the same 33.1% FPR. Updating every weight from one site's skewed data degrades the shared ranking. The low-rank restriction limits this degradation, but it does not produce a gain either.

Local objective. FedProx ($\mu{=}0.01$) gives no measurable gain over FedAvg: 52.4% vs. 50.5% recall and 19.0% vs. 18.3% FPR, both within one standard deviation. This is consistent with the already bounded local update, which limits client drift without a proximal term.

Training components. Removing FGSM training leaves clean accuracy essentially unchanged (86.78% vs. 86.47%). Accuracy under an FGSM perturbation ($\epsilon{=}0.05$) falls from 74.49% to 52.37%. Removing hard-example mining does not change Fuzzers recall (52.97%), but robustness becomes unstable across seeds (58.7$\pm$12.8%). We report mining as having no clear benefit.

Poisoning. One client uploads $-10\times$ its honest adapter (model replacement). Under FedAvg the global model collapses to near-chance accuracy (47.5$\pm$13.1%), alternating between labelling everything benign and everything malicious. With the adoption gate, every device rejected every poisoned global model in all rounds and seeds. However, the isolated devices keep training only on their own skewed data, and device 2's FPR drifts to 38.8%. Coordinate-wise median aggregation removes the attack at the server: 86.54$\pm$0.21% accuracy and 18.46$\pm$2.20% FPR, matching the clean runs. Median aggregation on clean data costs little (86.31% accuracy, 20.1% FPR; AUROC 0.959, identical to FedAvg). The two defences are complementary: the median protects the shared model, and the gate protects a device if the server is compromised.

D. The agentic decision layer

Action selection (Table VI). On UNSW-NB15, the cost-aware rule cuts missed attacks from 11.6% to 2.4% of attack flows and lowers the average cost per flow by 23%. The price is that it alerts on 31.5% of benign flows instead of 12.6%, which means more analyst workload. On ToN_IoT both rules are already near-perfect, and the cost rule halves missed attacks (0.92% $\rightarrow$ 0.39%).

Response and learning (Table VII; Fig. 4). Blocking per source IP magnifies false positives: one wrong verdict on a busy benign host cuts off all of its later traffic. With permanent blocks (configuration A) this drops 49.8% of benign ToN_IoT flows and 22.1% of benign UNSW-NB15 flows.

- Graded, self-expiring blocks (C) reduce this to 28.6% and 1.4%, without any analyst.
- Analyst rollback with learned exceptions (D) reduces it further, to 9.4% and 1.2%.
- Peer intelligence with retraction (E) costs a little benign traffic (12.5% and 2.1%) but stops more attacks. Attack flows that get through fall from 0.029% to 0.019% (ToN_IoT) and from 0.38% to 0.13% (UNSW-NB15) relative to the threshold-and-block baseline.

With an oracle analyst, the full agent therefore dominates the baseline on both axes and both datasets.

Imperfect analyst (Table VIII). In the next experiment, the analyst reviews only a random half of the blocks and is wrong 10% or 20% of the time. Benign traffic dropped stays well below the baseline (24.0% vs. 49.8% on ToN_IoT, 4.1% vs. 22.1% on UNSW-NB15 at 10% error). Attack leakage, however, grows sharply: 4.5% of ToN_IoT attack flows get through at 10% error and 11.8% at 20% error, against 0.083% for configuration C with no analyst at all. The cause is the learned exception. A single wrong rollback of an attacker grants it 30 minutes in which block verdicts are downgraded to alerts, and the retraction propagates to every peer. Shortening the exception to 5 minutes cuts the leakage by about two-thirds (4.5% $\rightarrow$ 1.5% on ToN_IoT and 3.1% $\rightarrow$ 1.1% on UNSW-NB15 at 10% error). It costs more benign blocking with an ideal analyst (12.5% $\rightarrow$ 18.6% on ToN_IoT). Analyst feedback is only worth enabling where analyst verdicts are reliable. Otherwise graded blocks alone (C) are the safer configuration.

VI. Limitations

- On the official UNSW-NB15 split the backbone reaches 87.7% accuracy, below the >96% often reported on random splits (97.9% on our own random validation split). Gradient-boosted trees on the same eight features reach 90.1%, so about 2.4 points of the gap are attributable to the Transformer itself; the rest is distribution shift between the official files.
- Federated adaptation did not improve class separation in our setting: its recall gain equals a threshold shift of the frozen model (Table IV), and UNSW FPR rises with every round. This held with one to three of four clients holding the new class and with 100 or 500 local steps per round, over four rounds. More clients, more rounds or other local objectives may differ.
- FedAvg weights clients by self-reported n_k. A malicious client can inflate its weight. The adoption gate limits but does not remove this.
- Device partitions are drawn from the same pool as pretraining, except device 1's Fuzzers rows. The (label-

TABLE V

ABLATIONS AFTER 4 FEDERATED ROUNDS, GLOBAL MODEL ON DEVICE 2 (MEAN ± SD OVER 3 SEEDS; EC2). PAYLOAD = RAW FLOAT32 BYTES UPLOADED PER DEVICE PER ROUND. ADV. ACC. = ACCURACY UNDER FGSM, ϵ=0.05, 5,000 TEST FLOWS.

Variant	Payload (B)	Fuzzers rec. (%)	UNSW acc. (%)	UNSW FPR (%)	Macro-F1	Adv. acc. (%)	ToN acc. (%)
LoRA r=1	2,835	51.25 ± 1.71	86.67 ± 0.13	18.26 ± 0.69	0.864 ± 0.001	73.82 ± 1.64	98.90 ± 0.04
LoRA r=2	5,403	52.42 ± 0.11	86.55 ± 0.14	18.66 ± 0.30	0.863 ± 0.001	73.50 ± 1.48	98.87 ± 0.01
LoRA r=4 (default)	10,515	50.48 ± 3.16	86.47 ± 0.04	18.28 ± 0.83	0.862 ± 0.001	74.49 ± 0.15	98.19 ± 1.09
LoRA r=8	20,763	59.12 ± 6.45	86.00 ± 0.56	21.24 ± 2.66	0.857 ± 0.007	75.21 ± 1.07	97.72 ± 0.97
Full-model FedAvg	106,155	78.04 ± 3.34	82.66 ± 0.40	33.08 ± 1.80	0.817 ± 0.005	59.13 ± 13.48	95.95 ± 2.51
w/o hard-example mining	10,515	52.97 ± 2.58	86.62 ± 0.02	18.65 ± 0.70	0.864 ± 0.000	58.71 ± 12.80	98.81 ± 0.05
w/o FGSM training	10,515	50.26 ± 0.15	86.78 ± 0.03	17.67 ± 0.12	0.866 ± 0.000	52.37 ± 0.37	99.11 ± 0.01
FedProx (μ=0.01)	10,515	52.42 ± 3.88	86.37 ± 0.10	18.98 ± 1.13	0.861 ± 0.001	74.56 ± 0.16	98.81 ± 0.01
Median aggregation	10,515	56.84 ± 3.00	86.31 ± 0.15	20.14 ± 1.02	0.860 ± 0.002	75.01 ± 0.89	98.21 ± 1.05
+ adoption gate	10,515	48.88 ± 4.95	86.47 ± 0.12	17.83 ± 1.43	0.863 ± 0.002	74.58 ± 0.34	98.18 ± 1.08
Poisoned client, FedAvg	10,515	76.20 ± 41.22	47.51 ± 13.08	81.25 ± 32.47	0.337 ± 0.031	44.79 ± 15.57	72.06 ± 7.36
Poisoned client, FedAvg + gate	10,515	88.56 ± 3.51	81.32 ± 0.69	38.82 ± 2.34	0.799 ± 0.009	76.29 ± 1.14	89.56 ± 0.31
Poisoned client, median	10,515	50.86 ± 6.05	86.54 ± 0.21	18.46 ± 2.20	0.863 ± 0.003	69.16 ± 13.43	98.67 ± 0.03
Fuzzers on 2 clients	10,515	51.15 ± 6.93	86.46 ± 0.22	18.47 ± 2.19	0.862 ± 0.003	74.36 ± 0.52	98.22 ± 1.02
Fuzzers on 3 clients	10,515	79.42 ± 1.40	82.72 ± 0.22	33.56 ± 0.83	0.818 ± 0.003	77.03 ± 0.27	98.78 ± 0.02
5× local steps	10,515	71.41 ± 11.17	84.15 ± 1.92	28.24 ± 6.83	0.835 ± 0.022	53.93 ± 1.34	97.10 ± 0.08
3 clients + 5× steps	10,515	87.19 ± 4.38	67.04 ± 12.69	37.36 ± 1.44	0.665 ± 0.119	54.24 ± 2.40	97.63 ± 1.09

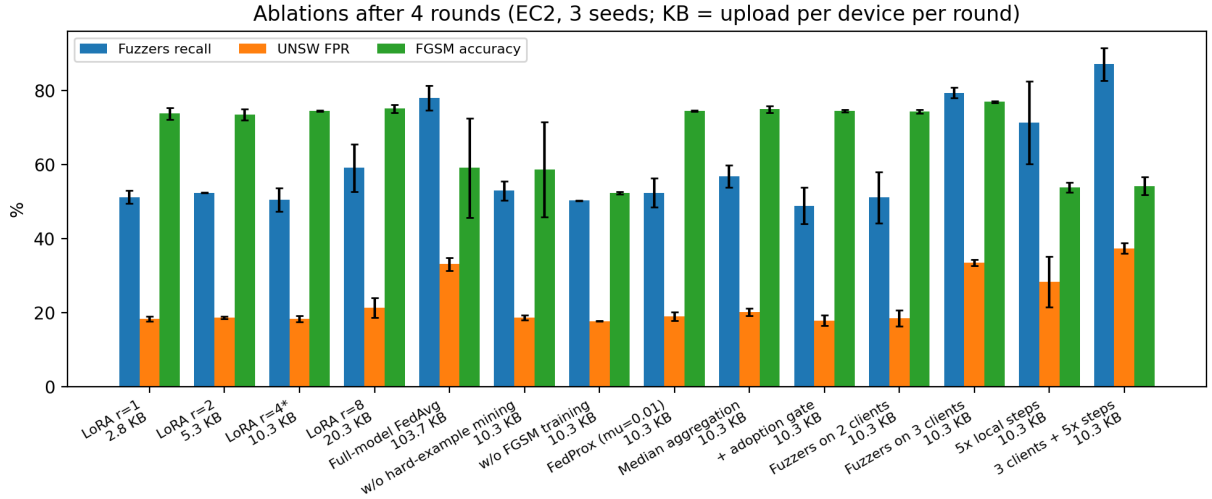


Fig. 3. Ablations after four rounds (3 seeds, EC2). Labels give the upload size per device per round. Poisoning results are in Table V.

TABLE VI

PER-FLOW DECISIONS OF THE FROZEN MODEL UNDER THE DEFAULT COST TABLE (DETERMINISTIC; SD = 0).

Data	Policy	False block (%)	Benign alert (%)	Attack missed (%)	Cost/flow
UNSW	Threshold ($p \geq 0.9$)	1.69	12.62	11.59	1.012
UNSW	Cost-aware (Bayes)	2.31	31.53	2.36	0.776
ToN	Threshold ($p \geq 0.9$)	1.24	0.69	0.92	0.152
ToN	Cost-aware (Bayes)	1.28	2.40	0.39	0.126

free) normalisation statistics were computed before the ToN_IoT test split was carved out. Recomputing them without the test rows shifts each feature mean by at most 0.058σ and each standard deviation by at most 2.4%; the official UNSW-NB15 test file was never included.

- The headline agent results assume an oracle analyst; with an imperfect analyst, attack leakage rises sharply (Table VIII). UNSW-NB15 agent results use synthetic host sessions. Live capture was validated only with loopback scans.
- Four clients only. Round experiments use a mocked cloud backend for reproducibility; the live backend was validated operationally, not for accuracy.

VII. CONCLUSION

Agentic FedGrid shows that low-rank federated adaptation and an agent that reasons about the cost and trustworthiness of its own actions can be combined on commodity cloud hardware. The measurements support four conclusions:

- Default-threshold recall overstates federated adaptation. In our setting, the recall gain on an unseen class is matched by moving the frozen model's threshold, and the class separation (AUROC) does not improve. This held with the class spread over up to three clients and with 5× longer local training. Federated IDS results should be reported threshold-free.

TABLE VII

FOUR-AGENT STREAM SIMULATION (3 SEEDS; EC2; ORACLE ANALYST). DROPPED = BENIGN FLOWS REFUSED BY AN ACTIVE BLOCK; UNBLOCKED = ATTACK FLOWS NOT BLOCKED OR DROPPED. TON_IOT USES REAL SOURCE IPS; UNSW-NB15 USES SYNTHETIC SESSIONS.

	ToN_IoT		UNSW-NB15	
Agent configuration	Benign dropped (%)	Attack unblocked (%)	Benign dropped (%)	Attack unblocked (%)
A: threshold rule, permanent block	49.79 ± 7.13	0.029 ± 0.008	22.05 ± 0.60	0.381 ± 0.044
B: cost-aware, permanent block	51.07 ± 6.75	0.028 ± 0.006	28.35 ± 0.62	0.285 ± 0.022
C: + graded blocks (TTL)	28.63 ± 5.67	0.083 ± 0.021	1.42 ± 0.06	1.267 ± 0.119
D: + analyst rollback & exceptions	9.40 ± 0.70	0.072 ± 0.013	1.21 ± 0.02	1.113 ± 0.160
E: + peer intel w/ retraction (full)	12.50 ± 1.91	0.019 ± 0.005	2.14 ± 0.08	0.132 ± 0.032

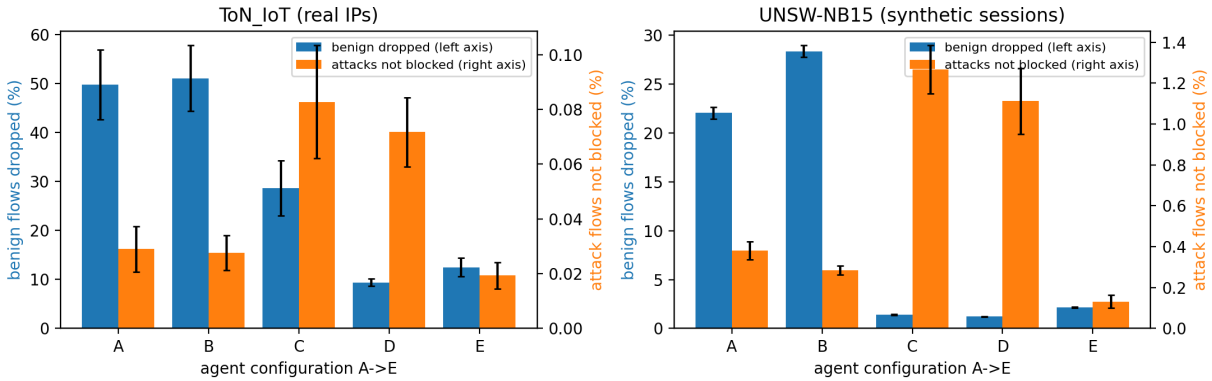


Fig. 4. Agent configurations A (threshold, permanent block) to E (full agent): benign flows dropped and attack flows not blocked, four agents, 3 seeds.

TABLE VIII

SENSITIVITY OF THE FULL AGENT TO THE ANALYST (3 SEEDS; EC2). THE ANALYST REVIEWS A RANDOM 50% OF BLOCKS AND GIVES THE WRONG VERDICT WITH THE STATED PROBABILITY; A WRONG VERDICT ON AN ATTACKER ROLLS ITS BLOCK BACK AND GRANTS IT THE LEARNED EXCEPTION.

	ToN_IoT		UNSW-NB15	
Configuration	Benign dropped (%)	Attack unblocked (%)	Benign dropped (%)	Attack unblocked (%)
C: no analyst	28.63 ± 5.67	0.083 ± 0.021	1.42 ± 0.06	1.267 ± 0.119
E: oracle analyst, 30-min exception	12.50 ± 1.91	0.019 ± 0.005	2.14 ± 0.08	0.132 ± 0.032
E: 50% reviewed, 10% wrong	23.97 ± 4.54	4.543 ± 1.583	4.05 ± 0.32	3.067 ± 1.217
E: 50% reviewed, 20% wrong	31.77 ± 8.34	11.785 ± 3.186	4.13 ± 0.27	12.362 ± 1.731
E: oracle, 5-min exception	18.56 ± 1.58	0.018 ± 0.006	2.38 ± 0.10	0.132 ± 0.032
E: 10% wrong, 5-min exception	29.16 ± 4.45	1.474 ± 0.228	4.25 ± 0.14	1.063 ± 0.396
E: 20% wrong, 5-min exception	34.46 ± 6.44	4.893 ± 0.235	4.40 ± 0.36	3.049 ± 0.783

- Low-rank adaptation does not add discrimination here, but at the default local budget it preserves the backbone's ranking where full-model federation degrades it. Median aggregation together with verified adoption neutralises model-replacement poisoning.
- The main obstacle in edge intrusion detection is false positives, and per-host blocking magnifies them. Graded, self-expiring response removes most of this cost without any human input.
- Human feedback helps only when it is reliable. Mistaken rollbacks are amplified by learned exceptions and shared retractions, so their duration must be short.

Future work: federated objectives that change the representation rather than the threshold, such as contrastive or multi-class local objectives (more clients holding the class and longer local training did not help here); real analyst feedback; and temporal (multi-flow) models that reduce the per-flow false positives behind most of the costs measured here.

REFERENCES

[1] A. Vyas, P.-C. Lin, R.-H. Hwang, and M. Tripathi, "Privacy-preserving federated learning for intrusion detection in IoT environments: A survey," *IEEE Access*, vol. 12, pp. 127018–127050, 2024, doi: 10.1109/ACCESS.2024.3454211.

[2] P. V. Pandit, S. Bhushan, and P. V. Waje, "Implementation of intrusion detection system using various machine learning approaches with ensemble learning," in *Proc. Int. Conf. Advancement in Computation & Computer Technologies (InCACCT)*, 2023, pp. 468–472, doi: 10.1109/InCACCT57535.2023.10141704.

[3] M. R. Bayesh and M. S. Hossain, "An automated network intrusion detection and identification system by synergizing AI techniques," in *Proc. IEEE 3rd Int. Conf. Robotics, Automation, Artificial-Intelligence and Internet-of-Things (RAAICON)*, Dhaka, Bangladesh, 2024, pp. 203–206, doi: 10.1109/RAAICON64172.2024.10928543.

[4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[5] O. Aouedi, K. Piamrat, G. Muller, and K. Singh, "FLUIDS: Federated learning with semi-supervised approach for intrusion detection system," in *Proc. IEEE 19th Annu. Consumer Communications & Networking Conf. (CCNC)*, 2022, pp. 523–524, doi: 10.1109/CCNC49033.2022.9700632.

[6] R. Devi, N. Gomathi, S. Geerthik, B. Jayaram, P. Manikandaprabu, and S. Nachiappan, "Vision transformers-enhanced cybersecurity framework for intrusion detection," in *Proc. 10th Int. Conf. Science Technology Engineering and Mathematics (ICONSTEM)*, 2025, doi: 10.1109/ICONSTEM65670.2025.11374855.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[8] S. Panwar and H. Abdelrahman, "Agentic AI in cybersecurity: Review of autonomous threat detection and adaptive defense mechanisms," in *Proc. Int. Conf. Computer and Applications (ICCA)*, 2025, doi: 10.1109/ICCA66035.2025.11431003.

[9] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Military Communications and Information Systems Conf. (MilCIS)*, 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.

[10] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.

[11] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, and C. Miao, "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, 2020.

[12] S. F. Lameh, W. Noble, Y. Amannejad, and A. Afshar, "Analysis of federated learning as a distributed solution for learning on edge devices," in *Proc. Int. Conf. Intelligent Data Science Technologies and Applications (IDSTA)*, 2020, pp. 66–74.

[13] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet of Things Journal*, vol. 9, no. 4, pp. 2545–2554, 2022.

[14] P. Ruzafa-Alcázar *et al.*, "Intrusion detection based on privacy-preserving federated learning for the industrial IoT," *IEEE Trans. Industrial Informatics*, vol. 19, no. 2, pp. 1145–1154, 2023, doi: 10.1109/TII.2021.3126728.

[15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, 2020.

[16] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2020.

[17] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018.

[18] L. Lyu, H. Yu, X. Ma, C. Chen, L. Sun, J. Zhao, Q. Yang, and P. S. Yu, "Privacy and robustness in federated learning: Attacks and defenses," *IEEE Trans. Neural Networks and Learning Systems*, 2022.

[19] K. Hammar and R. Stadler, "Intrusion prevention through optimal stopping," *IEEE Trans. Network and Service Management*, vol. 19, no. 3, pp. 2333–2348, 2022.

[20] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4190–4197.

[21] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[22] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.